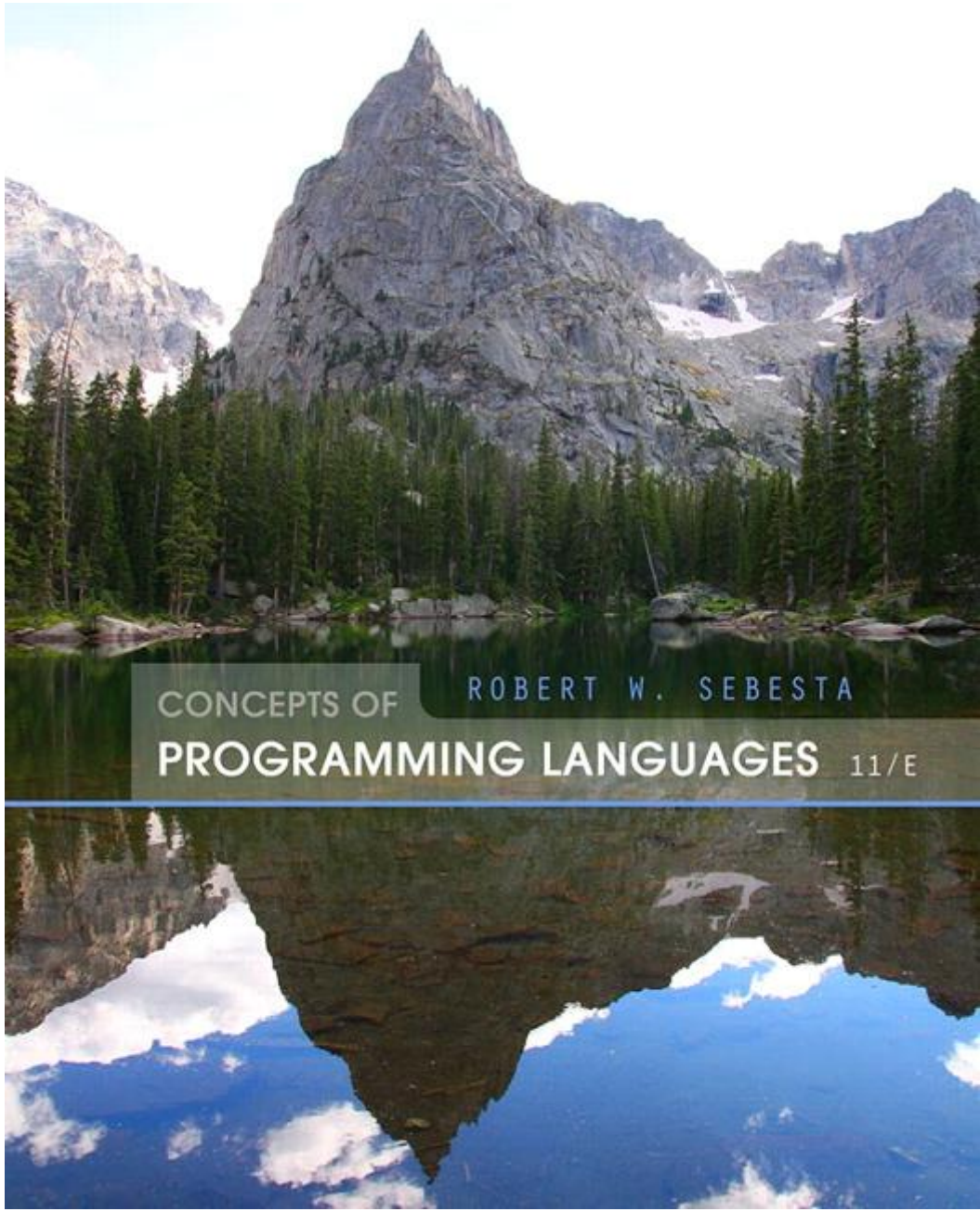
Bölüm 2

Başlıca Programlama Dillerinin Evrimi



ISBN 0-321-49362-1

Bölüm 2 Konular

- 2.1 Zuse'nin Plankalkülü (Zuse's Plankalkül)
- 2.2 Minimal Donanım Programlama: Sözde Kodlar
- 2.3 IBM 704 ve Fortran
- 2.4 Fonksiyonel Programlama: Lisp
- 2.5 Sofistike Olmaya Doğru İlk Adım: ALGOL 60
- 2.6 İş Kayıtlarının Bilgisayar Ortamına Aktarılması: COBOL
- 2.7 Zaman paylaşımının başlangıcı: Basic

Copyright © 2015 Pearson. All rights reserved.

Bölüm 2 Konular (Devam)

- 2.8 Herkes için her şey: PL/I
- 2.9 İki Erken Dinamik Dil: APL ve SNOBOL
- 2.10 Veri Soyutlamanın Başlangıcı: SIMULA 67
- 2.11 Ortogonal Tasarım: ALGOL 68
- 2.12 ALGOL'ların Bazı Erken Dönem Torunları
- 2.13 Mantığa Dayalı Programlama: Prolog
- 2.14 Tarihin En Büyük Tasarım Çalışması: Ada

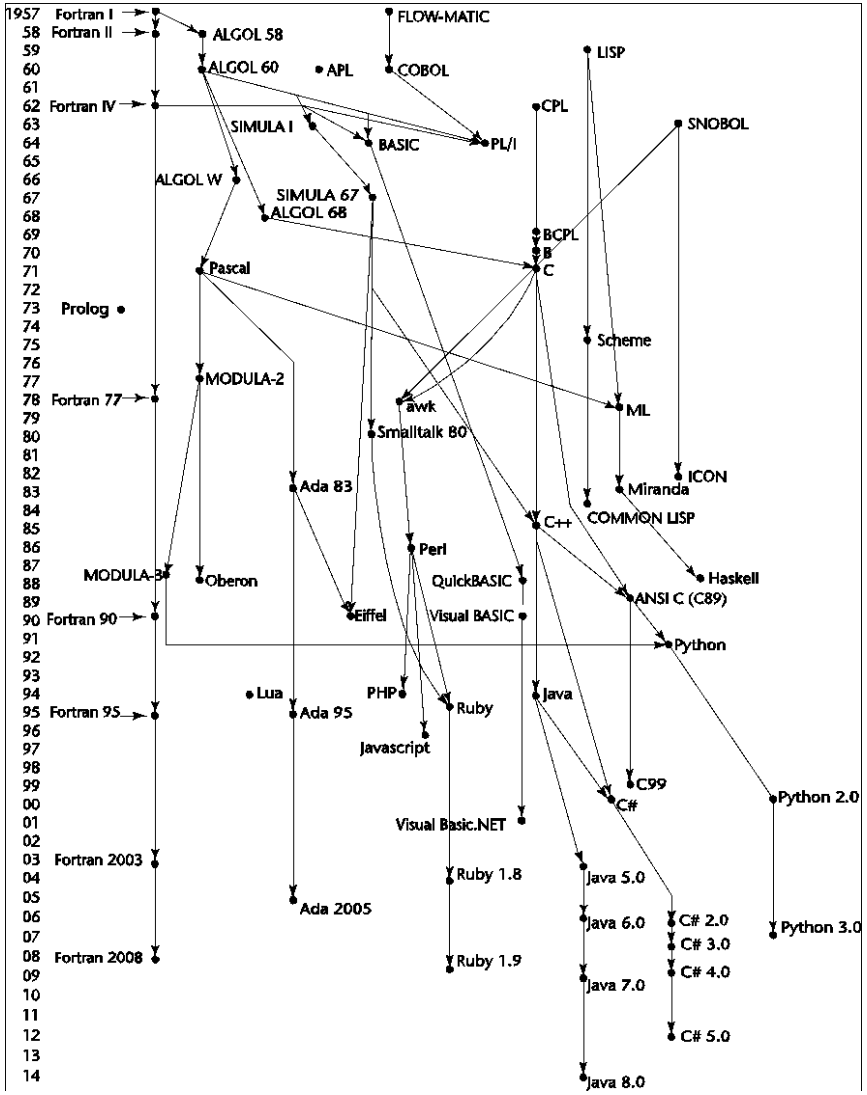
Copyright © 2015 Pearson. All rights reserved.

Bölüm 2 Konular (Devam)

- 2.15 Nesneye Yönelik Programlama: Smalltalk
- 2.16 Emir Kipi (Imperative) ve Nesneye Yönelik Özelliklerin Birleştirilmesi: C++
- 2.17 Emir Kipi (Imperative) Tabanlı Nesneye Yönelik Dil: Java
- 2.18 Scripting Dilleri
- 2.19 Amiral Gemisi .NET Dili: C#
- 2.20 İşaretleme(Markup)/Programlama Hibrit Dilleri

Copyright © 2015 Pearson. All rights reserved.

Ortak Dillerin Soy Ağacı



Copyright © 2015 Pearson. All rights reserved.

2.1 Zuse'nin Plankalkülü (Zuse's Plankalkül)

- 1945 yılında tasarlanmış, ancak 1972 yılına kadar yayınlanmamıştır.
- Hiç uygulanmadı
- Gelişmiş veri yapıları
  - kayan nokta, diziler, kayıtlar
- Sabitler
- Zuse, tek başına çalışarak, 1943 yılında doktora tezi için öneri olarak başlattığı Z4 için hesaplamaları ifade etmek üzere bir dil geliştirme çabasına girişti. Bu dile Zuse'nin Plankalkülü (Zuse's Plankalkül), yani program hesaplaması adı verildi.

Copyright © 2015 Pearson. All rights reserved.

2.1 Plankalkül Söz Dizimi (Syntax)

- A[4] + 1 ifadesini A[5]'e atamak için bir atama ifadesi

```
      |   A + 1 => A
V     |   4          5          (alt simgeler)
S     |   1.n        1.n        (veri türleri)
```

Copyright © 2015 Pearson. All rights reserved.

2.2 Minimal Donanım Programlama: Sözde Kodlar (Pseudocodes)

- Makine kodu kullanmanın nesi yanlıştı?
  - Zayıf okunabilirlik
  - Zayıf değiştirilebilirlik
  - İfade kodlaması zahmetliydi
  - Makine eksiklikleri - indeksleme veya kayan nokta yok

Copyright © 2015 Pearson. All rights reserved.

2.2.1 Sözde Kodlar: Short Code

- 1949 yılında Mauchly tarafından BINAC bilgisayarlar için geliştirilen “Short Code”
  - İfadeler kodlandı, soldan sağa doğru
  - İşlem örnekleri:

```
01 -    06 abs value    1n (n+2)nd power
02 )    07 +            2n (n+2)nd root
03 =    08 pause        4n if <= n
04 /    09 (            58 print and tab
```

Copyright © 2015 Pearson. All rights reserved.

2.2.2 Sözde Kodlar: Speedcoding

- 1954 yılında Backus tarafından IBM 701 için geliştirilen Speedcoding
  - Aritmetik ve matematik fonksiyonları için sözde işlemler
  - Koşullu ve koşulsuz dallanma
  - Dizi erişimi için otomatik artış registerları
  - Yavaş!
  - Kullanıcı programı için sadece 700 kelime bırakılmış

Copyright © 2015 Pearson. All rights reserved.

2.2.3 Sözde Kodlar: İlgili Sistemler

- UNIVAC Derleme Sistemi
  - Grace Hopper liderliğindeki bir ekip tarafından geliştirildi.
  - Makine koduna genişletilmiş sözde kod.
- David J. Wheeler (Cambridge Üniversitesi)
  - mutlak adresleme sorununu çözmek için yeniden konumlandırılabilir adres bloklarını kullanma yöntemi geliştirdi.

Copyright © 2015 Pearson. All rights reserved.

2.3 IBM 704 ve Fortran

- Fortran 0: 1954 - uygulanmadı
- Fortran I:1957
  - Dizin registerları ve kayan nokta donanımı olan yeni IBM 704 için tasarlanmıştır.

    - Bu, derlenmiş programlama dillerinin ortaya çıkmasına neden oldu, çünkü yorumlama maliyetini gizleyecek bir yer yoktu (kayan noktalı yazılım yoktu).

  - Geliştirme ortamı
    - Bilgisayarlar küçüktü ve güvenilmezdi.
    - Uygulamalar bilimseldi.
    - Programlama metodolojisi veya araçları yok.
    - Makine verimliliği en önemli endişe konusuydu.

Copyright © 2015 Pearson. All rights reserved.

2.3.2 Fortran'ın Tasarım Süreci

- Çevrenin Fortran I tasarımına etkisi
  - Dinamik depolama gerektirmez
  - İyi dizi işleme ve sayma döngüleri gerekir
  - Dize işleme, ondalık aritmetik veya güçlü giriş/çıkış (iş yazılımları için) yok

Copyright © 2015 Pearson. All rights reserved.

2.3.3 Fortran I Genel Bakış

- Fortran'ın ilk uygulanan versiyonu
  - İsimler en fazla altı karakterden oluşabilir.
  - Test sonrası sayım döngüsü (DO)
  - Biçimlendirilmiş I/O
  - Kullanıcı tanımlı alt programlar
  - Üç yönlü seçim ifadesi (aritmetik IF)
  - Veri tipini belirten ifadeler yok

Copyright © 2015 Pearson. All rights reserved.

2.3.3 Fortran I Genel Bakış (Devam)

- Fortran'ın ilk uygulanan versiyonu
  - Ayrı derleme yok
  - Derleyici, 18 çalışanın emekleri sonucunda 1957 yılının Nisan ayında piyasaya sürüldü
  - 400 satırdan uzun programlar, çoğunlukla 704'ün düşük güvenilirliği nedeniyle nadiren doğru şekilde derleniyordu
  - Kod çok hızlıydı
  - Hızla yaygın olarak kullanılmaya başlandı

Copyright © 2015 Pearson. All rights reserved.

2.3.4 Fortran II

- 1958 yılında dağıtılmıştır.
  - Bağımsız derleme
  - Hatalar düzeltilmiştir

Copyright © 2015 Pearson. All rights reserved.

2.3.5 Fortran IV

- 1960-62 yılları arasında gelişti.
  - Açık tip bildirimleri (type declarations)
  - Mantıksal seçim ifadesi
  - Alt program adları parametre olabilir
  - 1966 ANSI standardı

Copyright © 2015 Pearson. All rights reserved.

2.3.6 Fortran 77

- 1978 yılında yeni standart haline geldi
  - Karakter dizisi işleme
  - Mantıksal döngü kontrol ifadesi
  - **`IF-THEN-ELSE`** ifadesi

Copyright © 2015 Pearson. All rights reserved.

2.3.7 Fortran 90

- Fortran 77'den sonra en önemli değişiklikler
  - Modüller
  - Dinamik diziler
  - Pointers
  - Özyineleme (Recursion)
  - **`CASE`** ifadesi
  - Parametre türü kontrolü

Copyright © 2015 Pearson. All rights reserved.

2.3.8 Fortran'ın en son sürümleri

- Fortran 95 – nispeten küçük eklemeler ve bazı silmeler
- Fortran 2003 – OOP desteği, prosedür pointerları, C ile birlikte çalışabilirlik
- Fortran 2008 – yerel kapsamlar için bloklar, eş diziler, `Do Concurrent`

Copyright © 2015 Pearson. All rights reserved.

2.3.9 Fortran Değerlendirmesi

- Yüksek düzeyde optimize eden derleyiciler (90'dan önceki tüm sürümler)
  - Tüm değişkenlerin türleri ve depolama alanları çalışma zamanından önce sabitlenir.
- Bilgisayarların kullanım şeklini sonsuza dek dramatik bir şekilde değiştirdi.

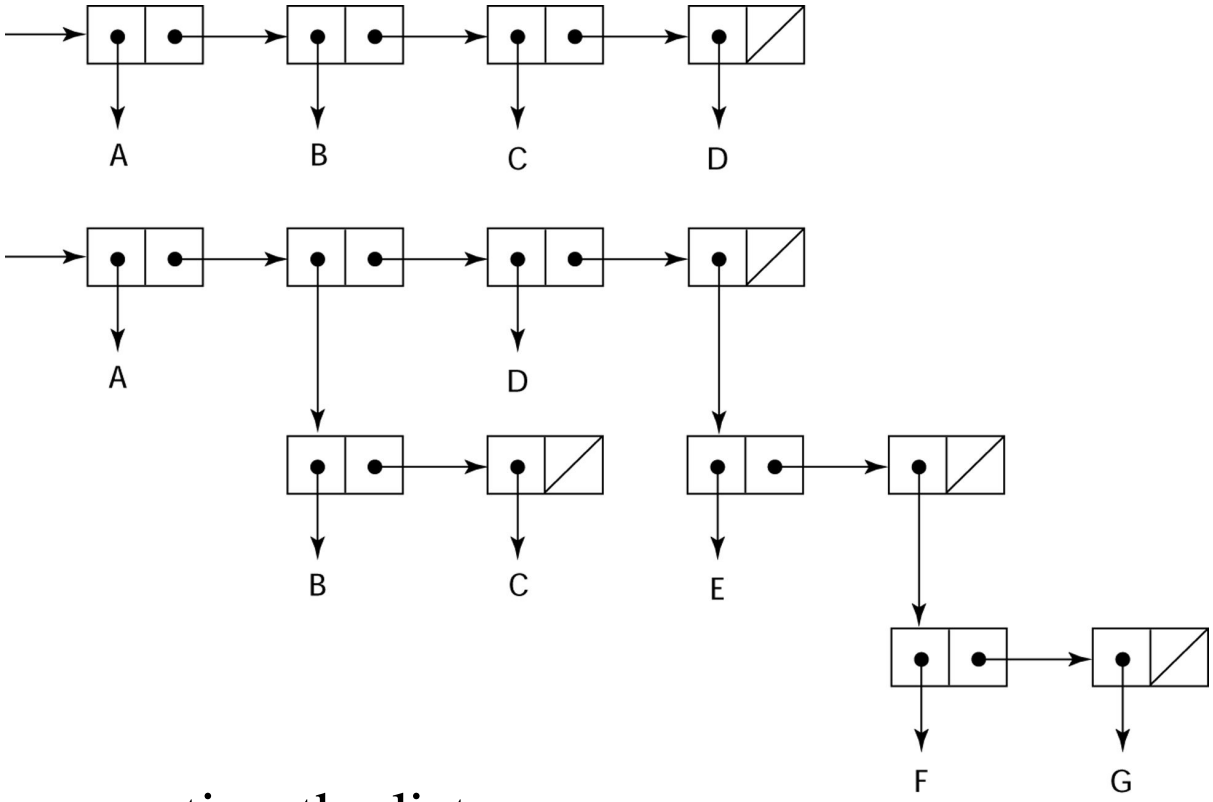
Copyright © 2015 Pearson. All rights reserved.

2.4 Fonksiyonel Programlama: Lisp

- LISt İşleme dili
  - MIT'de McCarthy tarafından tasarlanmıştır.
- AI araştırmaları için bir dil gerekiyordu
  - Verileri listelerde (diziler yerine) işleyebilecek
  - Sembolik hesaplama (sayısal değil)
- Sadece iki veri türü: atomlar ve listeler
- Söz dizimi lambda hesaplamasına (calculus) dayanmaktadır.

Copyright © 2015 Pearson. All rights reserved.

İki Lisp Listesinin Temsili



Representing the lists `(A B C D)` and `(A (B C) D (E (F G)))`

Copyright © 2015 Pearson. All rights reserved.

2.4.4 Lisp Değerlendirmesi

- Fonksiyonel programlamanın öncüsü
  - Değişkenlere veya atamalara gerek yok
  - Yineleme ve koşullu ifadelerle kontrol
- Hala yapay zeka için baskın dil
- Common Lisp ve Scheme, Lisp'in çağdaş lehçeleridir.
- ML, Haskell ve F# da fonksiyonel programlama dilleridir, ancak çok farklı söz dizimi kullanırlar.
-

Copyright © 2015 Pearson. All rights reserved.

2.4.4.1 Scheme

- 1970'lerin ortalarında MIT'de geliştirildi.
- Küçük
- Statik scoping'in yaygın kullanımı
- Birinci sınıf varlıklar olarak işlev görür
- Basit söz dizimi (ve küçük boyutu) onu eğitim uygulamaları için ideal hale getirir

Copyright © 2015 Pearson. All rights reserved.

2.4.4.2 Common Lisp

- Lisp'in çeşitli lehçelerinin özelliklerini tek bir dilde birleştirme çabası
- Büyük, karmaşık, endüstride bazı büyük uygulamalar için kullanılır

Copyright © 2015 Pearson. All rights reserved.

2.5 Sofistike Olmaya Doğru İlk Adım: ALGOL 60

- Geliştirme ortamı
  - FORTRAN, IBM 70x için (zar zor) piyasaya çıkmıştı.
  - Diğer birçok dil de geliştiriliyordu, hepsi belirli makineler için.
  - Taşınabilir dil yok; hepsi makineye bağlıydı.
  - Algoritmaları iletmek için evrensel bir dil yoktu.
- ALGOL 60, evrensel bir dil tasarlamak için yapılan çabaların sonucuydu.

Copyright © 2015 Pearson. All rights reserved.

2.5.1 Erken Tasarım Süreci

- ACM ve GAMM, tasarım için dört gün boyunca bir araya geldi (27 Mayıs - 1 Haziran 1958).

- Dilin amaçları
  - Matematiksel notasyona yakın olması
  - Algoritmaları tanımlamak için uygun olması
  - Makine koduna çevrilebilir olması

Copyright © 2015 Pearson. All rights reserved.

2.5.2 ALGOL 58

- Tür (type) kavramı resmileştirildi
- Names herhangi bir uzunlukta olabilir
- Arrays herhangi bir sayıda alt simgeye sahip olabilir
- Parameters moda göre ayrıldı (giriş ve çıkış)
- Subscripts parantez içine alındı
- Bileşik ifadeler (Compound statements) (**`begin ... end`**)
- İfade ayırıcı olarak noktalı virgül (semicolon)
- Atama operatörü :=
- **`if`** had an **`else-if`** clause
- I/O yok - “makineye bağımlı hale getirir”

Copyright © 2015 Pearson. All rights reserved.

2.5.3 ALGOL 58 Uygulaması

- Uygulanması amaçlanmamıştı, ancak bunun varyasyonları uygulandı (MAD, JOVIAL).
- IBM başlangıçta hevesli olsa da, 1959 ortasına kadar tüm desteği kesildi.

Copyright © 2015 Pearson. All rights reserved.

2.5.4 ALGOL 60 Genel Bakış

- Paris'te 6 günlük toplantıda değiştirilmiş ALGOL 58
- Yeni özellikler
  - Blok yapısı (yerel kapsam (scope))
  - İki parametre aktarım yöntemi
  - Alt program özyineleme
  - Yığın dinamik diziler
  - Hala I/O ve dize işleme yok

Copyright © 2015 Pearson. All rights reserved.

2.5.5 ALGOL 60 Değerlendirme

- Başarılar
  - 20 yılı aşkın bir süredir algoritmaları yayınlamanın standart yolu buydu.
  - Sonraki tüm emir dileri buna dayanmaktadır.
  - İlk makine bağımsız dil
  - Söz dizimi resmi olarak tanımlanan ilk dil (BNF)

Copyright © 2015 Pearson. All rights reserved.

2.5.5 ALGOL 60 Değerlendirme (Devam)

- Başarısızlıkları
  - Özellikle ABD'de hiç yaygın olarak kullanılmamıştır.
  - Sebepler
    - I/O ve karakter setinin eksikliği, programların taşınabilir olmamasına neden oldu.
    - Çok esnek - uygulaması zor
    - Fortran'ın Yerleşikleşmesi
    - Biçimsel söz dizimi açıklaması
    - IBM'den destek eksikliği

Copyright © 2015 Pearson. All rights reserved.

2.6 İş Kayıtlarının Bilgisayar Ortamına Aktarılması: COBOL

- Geliştirme ortamı
  - UNIVAC, FLOW-MATIC kullanmaya başlamıştı.
  - ABD Hava Kuvvetleri AIMACO'yu kullanmaya başlamıştı.
  - IBM, COMTRAN'ı geliştiriyordu.

Copyright © 2015 Pearson. All rights reserved.

2.6.1 COBOL Tarihsel Arka Plan

- FLOW-MATIC temel alındı
- FLOW-MATIC özellikleri
  - 12 karaktere kadar isimler, gömülü tire işaretleri ile
  - Aritmetik operatörlerin İngilizce isimleri (aritmetik ifadeler hariç)
  - Veriler ve kod tamamen ayrıydı
  - Her cümlenin ilk kelimesi bir fiil idi

Copyright © 2015 Pearson. All rights reserved.

2.6.2 COBOL Tasarım Süreci

- İlk Tasarım Toplantısı (Pentagon) - Mayıs 1959
- Tasarım hedefleri
  - Basit İngilizce gibi görünmelidir
  - Kullanımı kolay olmalı, bu daha az güçlü olacağı anlamına gelse bile
  - Bilgisayar kullanıcılarının tabanını genişletecek şekilde olmalı
  - Mevcut derleyici sorunlarından etkilenmemelidir
- Tasarım komitesi üyeleri, tamamı bilgisayar üreticileri ve Savunma Bakanlığı şubelerinden geliyordu.
- Tasarım Sorunları: aritmetik ifadeler? alt simgeler? Üreticiler arasındaki çekişmeler.

Copyright © 2015 Pearson. All rights reserved.

2.6.3 COBOL Değerlendirmesi

- Katkılar
  - Yüksek seviyeli bir dilde ilk makro özelliği
  - Hiyerarşik veri yapıları (kayıtlar)
  - İç içe seçim ifadeleri
  - Uzun isimler (30 karaktere kadar), tire ile
  - Ayrı veri bölümü

Copyright © 2015 Pearson. All rights reserved.

2.6.4 COBOL: Savunma Bakanlığı'nın Etkisi

- Savunma Bakanlığı tarafından istenen ilk dil
  - Savunma Bakanlığı olmasaydı başarısız olurdu
- Hala en yaygın kullanılan iş uygulamaları dili

Copyright © 2015 Pearson. All rights reserved.

2.7 Zaman Paylaşımının Başlangıcı: Basic

- Dartmouth'daki Kemeny & Kurtz tarafından tasarlanmıştır.
- Tasarım Hedefleri:
  - Fen bilimleri öğrencisi olmayanlar için öğrenmesi ve kullanması kolaydır
  - "Hoş ve dostça" olmalı
  - Ödevler için hızlı geri dönüş
  - Ücretsiz ve özel erişim
  - Kullanıcı zamanı bilgisayar zamanından daha önemlidir
- Güncel popüler lehçe: Visual Basic
- Zaman paylaşımı ile ilk yaygın olarak kullanılan dil

Copyright © 2015 Pearson. All rights reserved.

2.8 Herkes için her şey: PL/I

- IBM ve SHARE tarafından tasarlanmıştır.
- 1964 yılında bilgisayar durumu (IBM'in bakış açısı)
  - Bilimsel hesaplama
    - IBM 1620 ve 7090 bilgisayarları
    - FORTRAN
    - SHARE kullanıcı grubu
  - İşletme bilişimi
    - IBM 1401, 7080 bilgisayarları
    - COBOL
    - GUIDE kullanıcı grubu

Copyright © 2015 Pearson. All rights reserved.

2.8.1 PL/I: Arka plan

- 1963 yılına kadar
  - Bilimsel kullanıcılar, COBOL'da olduğu gibi daha ayrıntılı I/O'ya ihtiyaç duymaya başladı; iş kullanıcıları ise MIS için kayan nokta ve dizilere ihtiyaç duymaya başladı.
  - Birçok mağaza iki tür bilgisayar, dil ve destek personeline ihtiyaç duyacak gibi görünüyordu - bu çok maliyetliydi.
- Bariz çözüm
  - Her iki tür uygulamayı da çalıştırabilecek yeni bir bilgisayar kurmaktı.
  - Her iki tür uygulamayı da gerçekleştirmek için yeni bir dil tasarlamaktı.

Copyright © 2015 Pearson. All rights reserved.

2.8.2 PL/I: Tasarım Süreci

- 3 X 3 Komitesi tarafından beş ayda tasarlanmıştır.
  - IBM'den üç üye, SHARE'den üç üye
- İlk konsept
  - Fortran IV'ün bir uzantısı
- Başlangıçta NPL (Yeni Programlama Dili, New Programming Language) olarak adlandırıldı.
- 1965 yılında adı PL/I olarak değiştirildi.

Copyright © 2015 Pearson. All rights reserved.

2.8.3 PL/I: Değerlendirme

- PL/I katkıları
  - İlk birim düzeyinde eşzamanlılık
  - İlk istisna işleme
  - Switch ile seçilebilir özyineleme
  - İlk işaretçi (pointer) veri türü
  - İlk dizi kesitleri
- Endişeler
  - Birçok yeni özellik kötü tasarlanmıştı.
  - Çok büyük ve çok karmaşık.

Copyright © 2015 Pearson. All rights reserved.

2.9 İki Erken Dinamik Dil: APL ve SNOBOL

- Dinamik tipleme ve dinamik depolama tahsisi ile karakterize edilir.
- Değişkenlerin türü tanımlanmamış
  - Bir değişken, bir değer atandığında bir tür kazanır
- Değer atandığında değişkene depolama alanı ayrılır.

Copyright © 2015 Pearson. All rights reserved.

2.9.1 APL: A Programming Language

- 1960 civarında Ken Iverson tarafından IBM'de bir donanım tanımlama dili olarak tasarlanmıştır.
  - Son derece ifade gücü yüksek (hem skalerler hem de çeşitli boyutlardaki diziler için çok sayıda operatör)
  - Programları okumak çok zor
- Hala kullanımda; minimum değişikliklerle

Copyright © 2015 Pearson. All rights reserved.

2.9.2 SNOBOL

- 1964 yılında Bell Labs'ta Farber, Griswold ve Polensky tarafından bir dizgi (string) işleme dili olarak tasarlanmıştır
- String desen eşleştirme için güçlü operatörler
- Alternatif dillerden daha yavaş (ve bu nedenle artık yazma editörleri için kullanılmıyor)
- Hala belirli metin işleme görevleri için kullanılıyor

Copyright © 2015 Pearson. All rights reserved.

2.10 Veri Soyutlamanın Başlangıcı: SIMULA 67

- Nygaard ve Dahl tarafından Norveç'te sistem simülasyonu için tasarlanmıştır.
- ALGOL 60 ve SIMULA I temel alınmıştır.
- Birincil Katkılar
  - Coroutines - bir tür alt program
  - Sınıflar, nesneler ve kalıtım

Copyright © 2015 Pearson. All rights reserved.

2.11 Ortogonal Tasarım: ALGOL 68

- ALGOL 60'ın sürekli geliştirilmesinden kaynaklanmaktadır, ancak bu dilin bir üst kümesi değildir.
- Birçok yeni fikrin kaynağı (dil kendisi yaygın olarak kullanılmamış olsa da)
- Tasarım, ortogonalite kavramına dayanmaktadır.
  - Birkaç temel kavram ve birkaç birleştirme mekanizması

Copyright © 2015 Pearson. All rights reserved.

2.11.1 ALGOL 68 Değerlendirme

- Katkılar
  - Kullanıcı tanımlı veri yapıları
  - Referans türleri
  - Dinamik diziler (esnek diziler olarak adlandırılır)
- Yorumlar
  - ALGOL 60'tan daha az kullanım
  - Sonraki diller üzerinde, özellikle Pascal, C ve Ada üzerinde güçlü bir etkiye sahipti

Copyright © 2015 Pearson. All rights reserved.

2.12.1 Pascal - 1971

- Wirth (ALGOL 68 komitesinin eski üyesi) tarafından geliştirilmiştir.
- Yapısal programlamayı öğretmek için tasarlanmıştır.
- Küçük, basit, gerçekten yeni bir şey yok.
- En büyük etki programlama öğretiminde oldu.
  - 1970'lerin ortalarından 1990'ların sonlarına kadar, programlama öğretiminde en yaygın olarak kullanılan dildi.

Copyright © 2015 Pearson. All rights reserved.

2.12.2 C - 1972

- Sistem programlama için tasarlanmıştır (Bell Labs'ta Dennis Richie tarafından)
- Öncelikle BCLP ve B'den, ancak aynı zamanda ALGOL 68'den de gelişmiştir.
- Güçlü operatörler kümesi, ancak zayıf tip denetimi
- Başlangıçta UNIX üzerinden yayıldı
- Bir sistem dili olarak tasarlanmış olmasına rağmen, birçok uygulama alanında kullanılmıştır

Copyright © 2015 Pearson. All rights reserved.

2.13 Mantığa Dayalı Programlama: Prolog

- Comerauer ve Roussel (Aix-Marseille Üniversitesi) tarafından, Kowalski'nin (Edinburgh Üniversitesi) yardımıyla geliştirilmiştir.
- Biçimsel mantığa dayalı
- Prosedürel olmayan
- Verilen sorguların doğruluğunu çıkarmak için bir çıkarım süreci kullanan akıllı bir veri tabanı sistemi olarak özetlenebilir.
- Nispeten verimsiz
- Az sayıda uygulama alanı

Copyright © 2015 Pearson. All rights reserved.

2.14 Tarihin En Büyük Tasarım Çalışması: Ada

- Yüzlerce kişinin katıldığı, çok para ve yaklaşık sekiz yıl süren devasa bir tasarım çalışması
- Gereksinimlerin sırası (1975-1978)
  - (Strawman, Woodman, Tinman, Ironman, Steelman)
- İlk programcı Augusta Ada Byron'un adını vermişler

Copyright © 2015 Pearson. All rights reserved.

2.14.1 Ada Değerlendirme

- Katkılar
  - Paketler - veri soyutlama desteği
  - İstisna işleme (Exception handling) - ayrıntılı
  - Genel program birimleri
  - Eşzamanlılık - görevlendirme modeli aracılığıyla
- Yorumlar
  - Rekabetçi tasarım
  - O zamanlar yazılım mühendisliği ve dil tasarımı hakkında bilinen her şeyi içeriyordu
  - İlk derleyiciler çok zordu; ilk gerçekten kullanılabilir derleyici, dil tasarımı tamamlandıktan yaklaşık beş yıl sonra ortaya çıktı

Copyright © 2015 Pearson. All rights reserved.

2.14.2 Ada 95

- Ada 95 (1988'de başladı)
  - Tip türetme (derivation) yoluyla OOP desteği
  - Paylaşılan veriler için daha iyi kontrol mekanizmaları
  - Yeni eşzamanlılık özellikleri
  - Daha esnek kütüphaneler
- Ada 2005
  - Arayüzler (interfaces) ve senkronizasyon arayüzleri
- Popülerliği, Savunma Bakanlığı'nın artık kullanımını zorunlu kılmaması ve C++'ın popülerliği nedeniyle azaldı.

Copyright © 2015 Pearson. All rights reserved.

2.15 Nesneye Yönelik Programlama: Smalltalk

- Xerox PARC'ta, başlangıçta Alan Kay tarafından, daha sonra Adele Goldberg tarafından geliştirildi.
- Nesne yönelimli bir dilin ilk tam uygulaması (veri soyutlama, kalıtım ve dinamik bağlama)
- Grafik kullanıcı arayüzü tasarımında öncü oldu
- OOP tanıtıldı

Copyright © 2015 Pearson. All rights reserved.

2.16 Emirsel ve Nesneye Yönelik Programlamayı Birleştirme: C++

- 1980 yılında Stroustrup tarafından Bell Labs'ta geliştirilmiştir.
- C ve SIMULA 67'den geliştirilmiştir.
- SIMULA 67'den kısmen alınan nesne yönelimli programlama olanakları içerir.
- Kısmen hem prosedürel hem de nesne yönelimli programlamayı desteklediği için büyük ve karmaşık bir dildir.
- OOP ile birlikte hızla popülerlik kazandı.
- ANSI standardı Kasım 1997'de onaylanmıştır.
- Microsoft'un versiyonu: MC++
  - Properties, delegates, interfaces, no multiple inheritance

Copyright © 2015 Pearson. All rights reserved.

2.16.1 C++ İlgili bir OOP dili

- Objective-C (Brad Cox tarafından tasarlanmıştır – 1980'lerin başında)
  - Smalltalk tabanlı OOP için C plus desteği
  - Smalltalk'ın metot çağırma söz dizimini kullanır
  - Apple tarafından sistem programları için kullanılır

Copyright © 2015 Pearson. All rights reserved.

2.17 Emir Kipi Tabanlı Nesneye Yönelik Dil: Java

- 1990'ların başında Sun tarafından geliştirildi
  - C ve C++, gömülü elektronik cihazlar için yeterli değildi.
- C++ tabanlı
  - Önemli ölçüde basitleştirilmiştir (**struct**, **union**, **enum**, pointer aritmetiği ve C++'ın atama zorlamalarının yarısı dahil değildir).
  - **Yalnızca** OOP'yi destekler
  - Referans vardır, ancak pointer yoktur
  - Appletler ve bir tür eşzamanlılık desteği içerir.

Copyright © 2015 Pearson. All rights reserved.

2.17.1 Java Değerlendirme

- C++'ın birçok güvenli olmayan özelliğini ortadan kaldırdı
- Eşzamanlılığı (concurrency) destekler
- Appletler, GUI'ler ve veri tabanı erişimi için kütüphaneleri destekler
- Taşınabilir: Java Sanal Makine (JVM) kavramı, JIT derleyiciler
- Web programlamada yaygın olarak kullanılır
- Önceki dillerden daha hızlı bir şekilde yaygınlaşmıştır
- En son sürüm, 8, 2014 yılında yayınlandı

Copyright © 2015 Pearson. All rights reserved.

2.18 Web için Scripting Dilleri

- Perl
  - Larry Wall tarafından tasarlanmış — ilk olarak 1987'de piyasaya sürülmüştür
  - Değişkenler statik olarak türlendirilir ancak örtük olarak bildirilir
  - Değişkenin adının ilk harfiyle gösterilen üç farklı ad alanı (namespaces)
  - Güçlü, ancak biraz tehlikeli
  - Web'de CGI (Common Gateway Interface) programlama için yaygın olarak kullanılmaya başlandı
  - UNIX sistem yönetimi dilinin yerine de kullanılır
- JavaScript
  - Netscape'te başladı, ancak daha sonra Netscape ve Sun Microsystems'ın ortak girişimi haline geldi
  - Dinamik HTML belgeleri oluşturmak için sıklıkla kullanılan, istemci tarafında HTML'ye gömülü bir komut dosyası dili
  - Tamamen yorumlanmış (Purely interpreted)
  - Sadece benzer söz dizimi ile Java ile ilişkili
- PHP
  - PHP: Rasmus Lerdorf tarafından tasarlanan Hypertext Preprocessor
  - Sunucu tarafında HTML'ye gömülü bir komut dosyası dili olup, genellikle web üzerinden form işleme ve veri tabanı erişimi için kullanılır.
  - Tamamen yorumlanmış (Purely interpreted)

Copyright © 2015 Pearson. All rights reserved.

2.18 Web için Scripting Dilleri (Devam)

- Python
  - Bir OO yorumlanmış scripting dili
  - Tip kontrolü yapılmış ancak dinamik olarak tipleştirilmiş
  - CGI programlama ve form işleme için kullanılır
  - Dinamik olarak türlendirilmiş, ancak tür denetimi yapılmış
  - Lists, tuples ve hashs destekler
- Ruby
  - Japonya'da Yukihiro Matsumoto (namı diğer “Matz”) tarafından tasarlanmıştır.
  - Perl ve Python'un yerine geçmek üzere bu sürece başladı
  - Saf nesne yönelimli bir scripting dili
    - Tüm veriler nesnelerdir
  - Çoğu operatör, kullanıcı kodu tarafından yeniden tanımlanabilen metotlar olarak uygulanır.
  - Tamamen yorumlanmış (Purely interpreted)

Copyright © 2015 Pearson. All rights reserved.

2.18 Web için Scripting Dilleri (Devam)

- Lua
  - Bir OO yorumlanmış (interpreted) scripting dili
  - Tip kontrolü yapılmış ancak dinamik olarak tipleştirilmiş
  - CGI programlama ve form işleme için kullanılır
  - Dinamik olarak türlendirilmiş, ancak tür denetimi yapılmış
  - Tek bir veri yapısı olan tablo ile listeleri, tuple'ları ve hash'leri destekler
  - Kolayca genişletilebilir

Copyright © 2015 Pearson. All rights reserved.

2.19 Amiral Gemisi .NET Dili: C#

- .NET geliştirme platformunun bir parçası (2000)
- C++, Java ve Delphi tabanlı
- Pointerlar, delegate'ler, properties, enumeration türleri, sınırlı bir tür dinamik tipleme ve anonim türleri içerir.
- Hızla gelişiyor

Copyright © 2015 Pearson. All rights reserved.

2.20 Markup/Programlama Hibrit Dilleri

- XSLT
  - eXtensible Markup Language (XML): bir meta işaretleme dili
  - eXtensible Stylesheet Language Transformation (XSTL), XML belgelerini görüntüleme için dönüştürür
  - Programlama yapıları (örneğin, döngü)
- JSP
  - Java Server Pages: dinamik Web belgelerini destekleyen bir dizi teknoloji
  - JSP kütüphanesi olan JSTL, HTML öğeleri biçiminde programlama yapıları içerir

Copyright © 2015 Pearson. All rights reserved.

Özet

- Bir dizi önemli programlama dilinin geliştirilmesi, geliştirme ortamı ve değerlendirilmesi
- Dil tasarımında güncel konulara bakış açısı

Copyright © 2015 Pearson. All rights reserved.